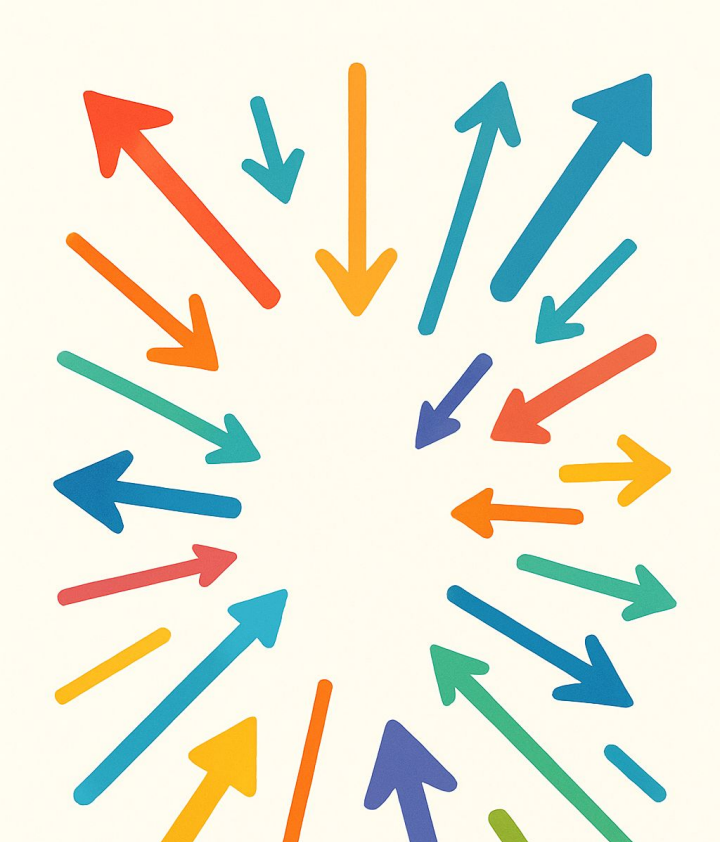
Les pointeurs



claranet®

Formation Go 2026

Pointeurs: quand les utiliser

- ✅ Modifier l'état d'un objet lourd dans une fonction/méthode
- ✅ Harmoniser un méthode set
- ✅ Liste chaînée et arbres
- ❌ Mutation d'une chaîne de caractères
- ❌ Mutation d'un ticker
- ❌ Utilisation dans plusieurs goroutines

Création

```
var t *T
```

```
t := new(T)
```

Création: var

```
msg := "my text" // création d'une valeur
var s *string // création d'une variable de type pointeur
s = &msg // affectation d'une valeur à un pointeur
fmt.Printf("value: %v - pointer value: %v - type: %T", s, *s, s)
// output: value: 0xc000012070 - pointer value: my text - type: *string
```

Création: new

```
msg := "my text" // création d'une valeur
s := new(string) // création d'une variable de type pointeur
s = &msg // affectation d'une valeur à un pointeur
fmt.Printf("value: %v - pointer value: %v - type: %T", s, *s, s)
// output: value: 0xc000012070 - pointer value: my text - type: *string
```

Création: new

```go
s1 := new(string)
fmt.Printf("value: %v - pointer value: \"%v\" - type: %T\n", s1, *s1, s1)
// output: value: 0xc000012070 - pointer value: "" - type: *string

var s2 *string
if s2 == nil { // si il n'y a pas de vérification le programme Panic
     s2 = s1
}
fmt.Printf("value: %v - pointer value: \"%v\" - type: %T\n", s2, *s2, s2)
// output: value: 0xc000012070 - pointer value: "" - type: *string
```

Création: new

```go
s1 := new(string)
fmt.Printf("value: %v - type: %T\n", s1, s1)
// output: value: 0xc000012070 - type: *string

var s2 *string
fmt.Printf("value: %v - type: %T\n", s2, s2)
// output: value: <nil> - type: *string
```

Création : valeur zéroifié

```go
s1 := new(string)
fmt.Printf("value: %v - pointer value: \"%v\" - type: %T\n", s1, *s1, s1)
// output: value: 0xc000012070 - pointer value: "" - type: *string

s2 := new(int)
fmt.Printf("value: %v - pointer value: %v - type: %T\n", s2, *s2, s2)
// output: value: 0xc000010058 - pointer value: 0 - type: *int

s3 := new(bool)
fmt.Printf("value: %v - pointer value: %v - type: %T\n", s3, *s3, s3)
// output: value: 0xc000010060 - pointer value: false - type: *bool
```

Les signes

Les signes : *

```
msg := "my text"

// Permet de créer une variable de type pointeur
var s *string

// Permet de déréférencer la valeur pointeur pour lui affecter une valeur
*s = msg

// Permet faire référence à un type pointeur dans une signature de fonction
func DoSomething(value *string){
    . . .
}
```

Les signes : &

```
// Donne la référence pointeur
msg := "my text"
var s *string = &msg // Ici on affecte une valeur pointeur au type *string

// Permet d'afficher la valeur pointeur
fmt.Printf("value: %v", &msg) // output: value: 0xc000012070
fmt.Printf("value: %s", &s) // output: value: 0xc000012070

// Permet faire référence à une valeur pointeur
func DoSomething() *string {
   s := "test text"
   return &s
}
```

Modification

Modification: valeur d'une référence

```go
msg := "my text"
var s *string = &msg

// Permet d'afficher la valeur
fmt.Printf("value: %v", msg) // output: value: my text
fmt.Printf("value: %s", *s) // output: value: my text

// On modifie la valeur d'origine
msg = "Mutate"

// Les deux valeurs sont modifiés
fmt.Printf("value: %v", msg) // output: value: Mutate
fmt.Printf("value: %s", *s) // output: value: Mutate
```

Modification: à l'extérieur

```go
func ModifyStr(str *string){
    *str = strings.ToUpper(*str)
}

. . .

// in main function
msg := "my text"

ModifyStr(&msg)

fmt.Printf("value: %v", msg) // output: value: MY TEXT
```